

Lecture Notes on Batch Normalization

A Complete Numerical Example

Dr. Adnan Arefeen
North South University

The full flow is:

$$X \rightarrow Z = W^T X + B \rightarrow H = \text{ReLU}(Z) \rightarrow \hat{H} \rightarrow Y = \gamma \hat{H} + \beta$$

Question:

Suppose we have:

Number of input features = 2

Number of samples = 3

Number of neurons = 2

We use the convention that each column of X represents one input sample.
The input matrix is:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 0 \end{bmatrix}$$

Therefore, the three input samples are:

$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad x^{(2)} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad x^{(3)} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

The weight matrix is:

$$W = \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}$$

Therefore,

$$W^T = \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}$$

The bias vector is:

$$b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Since we have three samples, the bias is repeated across all samples:

$$B = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

We use ReLU activation:

$$H = \text{ReLU}(Z)$$

For batch normalization, let:

$$\gamma = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \beta = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

1 Step 1: Compute the Linear Output

The linear output is computed as:

$$Z = W^T X + B$$

Substituting the matrices:

$$Z = \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Now compute each neuron output separately.

For neuron 1:

$$z_1^{(1)} = 1(1) + 2(2) + 0 = 5$$

$$z_1^{(2)} = 1(2) + 2(1) + 0 = 4$$

$$z_1^{(3)} = 1(3) + 2(0) + 0 = 3$$

For neuron 2:

$$z_2^{(1)} = 2(1) + (-1)(2) + 1 = 1$$

$$z_2^{(2)} = 2(2) + (-1)(1) + 1 = 4$$

$$z_2^{(3)} = 2(3) + (-1)(0) + 1 = 7$$

Therefore,

$$Z = \begin{bmatrix} 5 & 4 & 3 \\ 1 & 4 & 7 \end{bmatrix}$$

2 Step 2: Apply the Activation Function

We now apply the ReLU activation function:

$$H = \text{ReLU}(Z)$$

where

$$\text{ReLU}(x) = \max(0, x)$$

Since all values in Z are positive, ReLU does not change the values. Therefore,

$$H = \begin{bmatrix} 5 & 4 & 3 \\ 1 & 4 & 7 \end{bmatrix}$$

This activation matrix H is the input to batch normalization.

3 Step 3: Compute Batch Mean

Batch normalization computes the mean for each neuron across the batch.

For neuron 1:

$$\mu_1 = \frac{5 + 4 + 3}{3} = 4$$

For neuron 2:

$$\mu_2 = \frac{1 + 4 + 7}{3} = 4$$

Therefore,

$$\mu = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

4 Step 4: Compute Batch Variance

For neuron 1:

$$\sigma_1^2 = \frac{(5 - 4)^2 + (4 - 4)^2 + (3 - 4)^2}{3}$$

$$\sigma_1^2 = \frac{1 + 0 + 1}{3} = \frac{2}{3}$$

For neuron 2:

$$\sigma_2^2 = \frac{(1 - 4)^2 + (4 - 4)^2 + (7 - 4)^2}{3}$$

$$\sigma_2^2 = \frac{9 + 0 + 9}{3} = 6$$

Therefore,

$$\sigma^2 = \begin{bmatrix} \frac{2}{3} \\ 6 \end{bmatrix}$$

5 Step 5: Normalize the Activations

Batch normalization uses the formula:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

For simplicity, we ignore ϵ .

For neuron 1:

$$\hat{h}_1^{(1)} = \frac{5 - 4}{\sqrt{2/3}} \approx 1.225$$

$$\hat{h}_1^{(2)} = \frac{4 - 4}{\sqrt{2/3}} = 0$$

$$\hat{h}_1^{(3)} = \frac{3 - 4}{\sqrt{2/3}} \approx -1.225$$

For neuron 2:

$$\hat{h}_2^{(1)} = \frac{1 - 4}{\sqrt{6}} \approx -1.225$$

$$\hat{h}_2^{(2)} = \frac{4 - 4}{\sqrt{6}} = 0$$

$$\hat{h}_2^{(3)} = \frac{7 - 4}{\sqrt{6}} \approx 1.225$$

Therefore,

$$\hat{H} \approx \begin{bmatrix} 1.225 & 0 & -1.225 \\ -1.225 & 0 & 1.225 \end{bmatrix}$$

6 Step 6: Scale and Shift

Finally, batch normalization applies learnable parameters γ and β :

$$Y = \gamma \hat{H} + \beta$$

Here,

$$\gamma = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \beta = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

For neuron 1:

$$\gamma_1 = 1, \quad \beta_1 = 0$$

$$y_1^{(1)} = 1(1.225) + 0 = 1.225$$

$$y_1^{(2)} = 1(0) + 0 = 0$$

$$y_1^{(3)} = 1(-1.225) + 0 = -1.225$$

For neuron 2:

$$\gamma_2 = 2, \quad \beta_2 = 0.5$$

$$y_2^{(1)} = 2(-1.225) + 0.5 = -1.95$$

$$y_2^{(2)} = 2(0) + 0.5 = 0.5$$

$$y_2^{(3)} = 2(1.225) + 0.5 = 2.95$$

Therefore, the final batch-normalized output is:

$$Y \approx \begin{bmatrix} 1.225 & 0 & -1.225 \\ -1.95 & 0.5 & 2.95 \end{bmatrix}$$

Batch normalization normalizes the activation values of each neuron across the mini-batch. It then uses learnable parameters γ and β to scale and shift the normalized values.

Why Do We Need Batch Normalization?

We need **batch normalization** because, during training, the distribution of activations inside a neural network can change from one update to another. This can make training unstable and slow.

Suppose a layer produces activations like:

$$H = \begin{bmatrix} 100 & 0.01 \\ 120 & 0.02 \\ 90 & 0.015 \end{bmatrix}$$

Here, the first neuron has very large values, while the second neuron has very small values. This imbalance can make optimization harder because the next layer receives poorly scaled inputs.

Batch normalization fixes this by normalizing each neuron's activations across the mini-batch:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

After normalization, each neuron's activations have approximately:

$$\text{mean} \approx 0, \quad \text{variance} \approx 1$$

So the network receives more stable inputs from layer to layer.

Main Reasons for Using Batch Normalization

1. Faster training

Batch normalization keeps activations in a stable range, so gradient descent can move more efficiently.

2. More stable gradients

It helps reduce exploding or vanishing activation values, which makes backpropagation more stable.

3. Reduces sensitivity to initialization

Without batch normalization, poor weight initialization can make activations too large or too small. Batch normalization reduces this problem.

4. Acts like mild regularization

Since the mean and variance are computed from each mini-batch, there is some small noise during training. This can slightly reduce overfitting.

Simple Explanation

Batch normalization keeps layer outputs well-scaled and stable during training.